# File I/O & Streams in C

A deep guide — files, streams, and what sits under them

One topic, in depth. Covers the stream model and what a `FILE` really is, buffering and `setvbuf`, opening modes and their traps, text input (`fgetc`, `fgets`, `getline`), formatted and buffer output (`fprintf`, `snprintf`), binary I/O (`fread` / `fwrite`), seeking, end-of-file and error handling, and the file-descriptor layer underneath `stdio`. Every function entry gives the signature, what it does, an example, and the gotcha that bites people. Strings are a separate guide.

## 1. The stream model: what a `FILE` really is

In C you almost never touch a file directly. You talk to a **stream** — a `FILE` object the standard library manages for you. A `FILE *` is an *opaque handle*: you never look inside it, you just pass it to `stdio` functions. Under the hood it bundles three things:

- a connection to the actual file (a low-level **file descriptor** — section 9),
- a **buffer** in memory, so the program isn't making a system call for every byte,
- bookkeeping: the current **position**, plus the **end-of-file** and **error** flags.

That buffer is the whole point of streams. Reading one byte at a time with `fgetc` doesn't hit the disk each time; `stdio` pulls a big chunk into the buffer and hands you bytes from it. This is why `stdio` is called *buffered I/O*, and it's the theme that explains half the surprises below.

Three streams are open before your `main` runs (from `<stdio.h>`):

- `stdin` — standard input
- `stdout` — standard output
- `stderr` — standard error

Everything here works on all three as well as on files you open yourself.

## 2. Opening and closing files

```
FILE *fopen(const char *path, const char *mode);
int   fclose(FILE *stream);
```

`fopen` returns a `FILE *`, or `NULL` **on failure** (file missing, no permission, out of descriptors). Never skip the check:

```
FILE *fp = fopen("data.txt", "r");
if (fp == NULL) {
    perror("data.txt");      // prints why: "data.txt: No such file..."
    return 1;
}
/* ... use fp ... */
fclose(fp);                  // flushes the buffer, then releases the file
```

**The modes:**

| Mode | Read | Write | Start | If missing | If it exists |
|---|---|---|---|---|---|
| `"r"` | yes | no | start | error | kept |
| `"w"` | no | yes | start | created | **truncated** |
| `"a"` | no | yes | end | created | kept |
| `"r+"` | yes | yes | start | error | kept |
| `"w+"` | yes | yes | start | created | **truncated** |
| `"a+"` | yes | yes | end (writes) | created | kept |

Add `b` for binary (`"rb"`, `"wb"`, `"r+b"`, ...). On Linux it changes nothing; on Windows it disables newline translation, so always use it for non-text data.

Two subtleties most tutorials skip:

- **`"w"` destroys an existing file the instant you open it.** Want to add to a file? Use `"a"`. Edit in place? Use `"r+"`.
- **On the `+` modes you can't freely switch between reading and writing.** After a read, you must call `fseek`, `fflush`, or `rewind` before your first write (and vice versa). Skipping this is undefined behavior — a classic "my writes landed in the wrong place" bug.

`fclose` flushes buffered output and returns `0`, or `EOF` if a write error surfaced while flushing — so on files you care about, check its return

too.

`freopen(path, mode, stream)` reopens an existing stream onto a new file — handy for redirecting `stdout`:

```
freopen("out.log", "w", stdout);   // printf now writes to out.log
```

## 2b. Buffering (the part tutorials skip)

This is where "it printed on screen but the file is empty" bugs come from. A stream has one of three buffering modes:

- **Fully buffered** — data moves only when the buffer fills. Default for regular files.
- **Line buffered** — data flushes at each newline. Default for `stdout` when it's a terminal.
- **Unbuffered** — every write goes out immediately. Default for `stderr`, so errors are never lost.

What you'll actually hit:

```
printf("working...");   // no '\n' -> may sit in the buffer, unseen
/* crash here -> "working..." never prints */
```

Add a newline (a line-buffered terminal flushes it) or call `fflush(stdout)` to force it out. `fclose` and a normal exit flush automatically; a crash or `_exit` does not.

You can set the mode yourself, right after `fopen` and before any I/O:

```
int setvbuf(FILE *fp, char *buf, int mode, size_t size);
// mode: _IOFBF (full), _IOLBF (line), _IONBF (none)

setvbuf(fp, NULL, _IONBF, 0);   // make fp unbuffered
```

This also explains interleaving: mix `printf` (buffered `stdout`) with `fprintf(stderr, ...)` (unbuffered) and the `stderr` text can appear *before* earlier `stdout` text. Flush `stdout` first if order matters.

## 3. Text I/O

**One character at a time:**

```
int fgetc(FILE *fp);          // returns the byte as an int, or EOF
int fputc(int c, FILE *fp);
```

```
int c;
while ((c = fgetc(fp)) != EOF)   // c MUST be int, not char
    putchar(c);
```

`fgetc` returns an `int`, not a `char`, so `EOF` (a negative value) stays distinct from every valid byte. Store it in a `char` and the EOF test breaks — a very common bug.

**A fixed-size line:**

```
char *fgets(char *buf, int size, FILE *fp);
```

Reads up to `size-1` characters or through a newline, always null-terminates, keeps the `'\n'` if it fit, returns `NULL` at EOF/error. Safe — but a line longer than your buffer arrives in pieces.

**A line of any length (POSIX `getline`):**

```
// needs _POSIX_C_SOURCE >= 200809L (or _GNU_SOURCE) before <stdio.h>
ssize_t getline(char **lineptr, size_t *n, FILE *fp);
```

`getline` allocates and grows the buffer for you, so it handles arbitrarily long lines — the thing `fgets` can't. Start with a `NULL` pointer and size `0`; you **own the buffer** and free it once:

```
char  *line = NULL;
size_t cap  = 0;
ssize_t len;
while ((len = getline(&line, &cap, fp)) != -1) {
    fputs(line, stdout);     // one line; len is its length
}
free(line);                  // free ONCE, after the loop
```

This is the idiomatic modern way to read a file line by line on Linux/macOS.

**Write a string:**

```c
int fputs(const char *s, FILE *fp);   // no newline added
```

## 4. Formatted I/O, and formatting into buffers

**To and from a stream** — `printf` / `scanf` with a `FILE *`:

```c
int fprintf(FILE *fp, const char *fmt, ...);
int fscanf(FILE *fp, const char *fmt, ...);
```

`fprintf` returns the number of characters written. `fscanf` returns the number of items successfully matched (or `EOF`) — **check it**, because a format mismatch stops it early:

```c
int a, b;
if (fscanf(fp, "%d %d", &a, &b) != 2) {
    /* the line wasn't two integers */
}
```

**Into a string buffer** — `sprintf` and the safe `snprintf`:

```c
int sprintf(char *buf, const char *fmt, ...);             // UNSAFE
int snprintf(char *buf, size_t size, const char *fmt, ...);
```

`sprintf` has no idea how big `buf` is and will overflow it — avoid it. `snprintf` writes at most `size-1` characters plus `'\0'`, and returns the length it *would* have written, so a return `>= size` means it truncated:

```c
char path[64];
int n = snprintf(path, sizeof(path), "%s/%s", dir, name);
if (n >= (int)sizeof(path)) {
    /* truncated -- path was too small */
}
```

## 5. Binary I/O

For raw bytes — structs, arrays, images — skip the text functions and use:

```c
size_t fread (void *ptr, size_t size, size_t count, FILE *fp);
size_t fwrite(const void *ptr, size_t size, size_t count, FILE *fp);
```

Both take an element `size` and a `count`, and **return the number of complete elements** actually read or written — not bytes. A short return means EOF or error, so loop or check:

```c
int nums[4] = {10, 20, 30, 40};

FILE *fp = fopen("nums.bin", "wb");
fwrite(nums, sizeof(int), 4, fp);
fclose(fp);

int back[4];
fp = fopen("nums.bin", "rb");
size_t got = fread(back, sizeof(int), 4, fp);   // got == 4 if all present
fclose(fp);
```

Two depth warnings when you dump structs straight to disk:

- **Padding:** compilers insert invisible padding bytes inside structs, so the on-disk size is `sizeof(struct)`, not the sum of the fields.
- **Portability:** a file written this way encodes *this machine's* endianness and padding. It reads back fine on the same setup, not necessarily on a different CPU. For files meant to travel, define an explicit byte format.

## 6. Positioning within a file

```c
int   fseek(FILE *fp, long offset, int whence);  // SEEK_SET/CUR/END
long  ftell(FILE *fp);                           // current position
void  rewind(FILE *fp);                          // start, clears error flag
```

`whence` is `SEEK_SET` (from start), `SEEK_CUR` (from here), or `SEEK_END` (from end). Classic use — measure a file, then read it whole:

```c
fseek(fp, 0, SEEK_END);     // go to end
long size = ftell(fp);      // position == byte count
rewind(fp);                 // back to start
```

Two caveats worth knowing:

- On **text** streams (especially Windows), `ftell` values aren't plain byte offsets — only use them to return somewhere with `fseek`, don't do arithmetic on them. Open in binary (`"rb"`) when you need exact byte positions.
- `ftell`/`fseek` use `long`, which can be too small for files over ~2 GB. For huge files use `fgetpos`/`fsetpos` with an `fpos_t`:

```c
fpos_t pos;
fgetpos(fp, &pos);   // save a position
/* ... */
fsetpos(fp, &pos);   // return to it
```

## 7. End-of-file and error handling

```c
int feof(FILE *fp);     // nonzero once a read has already hit the end
int ferror(FILE *fp);   // nonzero if a stream error occurred
```

**The single most common file-I/O bug** is looping on `feof`:

```c
while (!feof(fp)) {          // WRONG -- one extra garbage iteration
    fgets(line, sizeof(line), fp);
    printf("%s", line);
}
```

`feof` only turns true *after* a read has already failed by reaching the end, so the body runs one last time on stale data. **Always drive the loop off the read call's own return value:**

```c
while (fgets(line, sizeof(line), fp)) { /* ... */ }      // RIGHT
```

Then, after the loop, use `feof`/`ferror` to tell a clean end from a real error:

```c
if (ferror(fp)) perror("read error");
else if (feof(fp)) { /* reached end normally */ }
```

For failures from `fopen` and friends, `errno` holds the reason; `perror("context")` prints `context: <reason>`, and `strerror(errno)` gives the string. `clearerr(fp)` resets the EOF/error flags if you want to keep using the stream.

## 8. Streams vs file descriptors (under the hood)

Everything above is the **buffered** `stdio` layer. Underneath it sits the operating system's **file descriptor** interface — the raw `open`, `read`, `write`, `close` system calls (POSIX, from `<unistd.h>` and `<fcntl.h>`). A `FILE *` is a wrapper that adds a buffer on top of one file descriptor.

You can move between the two layers:

```c
int   fileno(FILE *fp);                // get the fd inside a stream
FILE *fdopen(int fd, const char *mode); // wrap an fd in a stream
```

When would you drop to raw descriptors? When you need no buffering, precise control, or things `stdio` doesn't model — sockets, pipes, `poll`/`select`, non-blocking I/O. For ordinary file reading and writing the buffered layer is both easier and faster at runtime (fewer system calls), which is why it's the default. `fopen` is portable ISO C; `open` is POSIX — useful to know which world a given call lives in. (This is the layer your HTTP server will eventually sit on.)

## 9. Worked examples

**Robust line-by-line read with full error handling:**

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    FILE *fp = fopen("input.txt", "r");
    if (!fp) { perror("input.txt"); return 1; }

    char  *line = NULL;
    size_t cap  = 0;
    while (getline(&line, &cap, fp) != -1)
        fputs(line, stdout);

    if (ferror(fp)) perror("read");
    free(line);
    fclose(fp);
    return 0;
}
```

**Fast file copy with a block buffer:**

```c
#include <stdio.h>

int main(void) {
    FILE *in  = fopen("src.bin", "rb");
    FILE *out = fopen("dst.bin", "wb");
    if (!in || !out) { perror("open"); return 1; }

    char   buf[4096];
    size_t n;
    while ((n = fread(buf, 1, sizeof(buf), in)) > 0)
        fwrite(buf, 1, n, out);

    fclose(in);
    fclose(out);
    return 0;
}
```

**Read an entire file into memory:**

```c
FILE *fp = fopen("data.bin", "rb");
fseek(fp, 0, SEEK_END);
long size = ftell(fp);
rewind(fp);

char *buf = malloc(size + 1);
fread(buf, 1, size, fp);
buf[size] = '\0';        // usable as a string too
fclose(fp);
/* ... use buf ... */
free(buf);
```

## 10. Best-practices checklist

- Check `fopen` for `NULL` every time, and `perror` the reason.
- Always `fclose` what you open; check its return on files you must not lose.
- Never loop on `feof()` — drive loops off the `fgets` / `getline` / `fread` / `fgetc` return value.
- Store `fgetc`'s result in an `int`, not a `char`.
- Prefer `getline` (any length) or `fgets` (bounded) over `scanf` / `gets` for input.
- Use `snprintf`, never `sprintf`.
- Use binary mode ( `"rb"` / `"wb"` ) for non-text data and exact byte positions.
- Remember `"w"` truncates; on `+` modes, `fseek` / `fflush` between a read and a write.
- Check `fread` / `fwrite` return counts; `fflush` when output order matters.

## References & further reading

- **cppreference — file input/output:** https://en.cppreference.com/w/c/io
- **Linux man pages (section 3):** `man 3 fopen`, `man 3 fread`, `man 3 getline`, `man 3 setvbuf`, `man 3 fseek` — or https://man7.org/linux/man-pages/
- **POSIX / low-level I/O:** `man 2 open`, `man 2 read`, `man 2 write`; `man 3 fdopen`
- **K&R, *The C Programming Language* (2nd ed.):** Chapter 7 (Input and Output), and sections 8.1–8.4 for the link down to system calls.